Group members
Sara Algarni
Shatha Alshaikh
Lujain Alqarni

Table of Contents

Table of Figures

Table of Tables

2 PHASE 1: PROJECT DESCRIPTION

2.1 Introduction

Maintaining university campuses and large facilities requires effective coordination to ensure that buildings, equipment, and services operate smoothly; however, many institutions still rely on manual reporting methods or scattered communication channels when handling maintenance requests, which often leads to delays, lost requests, unclear responsibilities, and limited visibility into task progress. These inefficiencies can cause minor issues to escalate into major problems, affecting safety, comfort, and overall service quality. To address these challenges, the **MainTrack** system is proposed as a smart maintenance tracking platform that digitizes and organizes the entire maintenance workflow by providing a centralized system for submitting requests, automatically assigning tasks, tracking progress in real time, and offering monitoring and reporting tools that help management evaluate performance and make informed decisions, ultimately improving efficiency, transparency, and the reliability of facility maintenance services.

2.2 Project objectives

The main objective of **MainTrack** is to improve the efficiency and organization of maintenance operations within universities and large facilities. Effective maintenance management plays a vital role in ensuring safety, comfort, and continuity of services. A well-structured system enhances communication between users and maintenance teams, reduces delays in handling issues, improves accountability, and ultimately contributes to a better environment for all facility users. Thus, this project will focus on:

- Providing a centralized digital platform for reporting maintenance issues.
- Automating request prioritization and technician assignment based on urgency, workload, and availability.
- Enabling real-time tracking of maintenance request status.
- Sending system notifications to keep users and staff informed about important updates.
- Supporting real-time fault detection through IoT devices or sensor-based monitoring where available.
- Offering an administrative dashboard for monitoring operations, managing users, and evaluating technician performance.

- Generating reports and analytics to measure response times, resolution rates, and overall maintenance efficiency.
- Ensuring the system is user-friendly and accessible to individuals with different technical skill levels.
- Maintaining accurate documentation and history of maintenance activities for transparency and accountability.

2.3 Project organization

This diagram presents the overall structure of the **MainTrack** project, showing its main phases Project Description, Project Management, Requirements, Design, and Testing and the key components under each phase. It provides a clear and simple visual overview of how the project is organized.

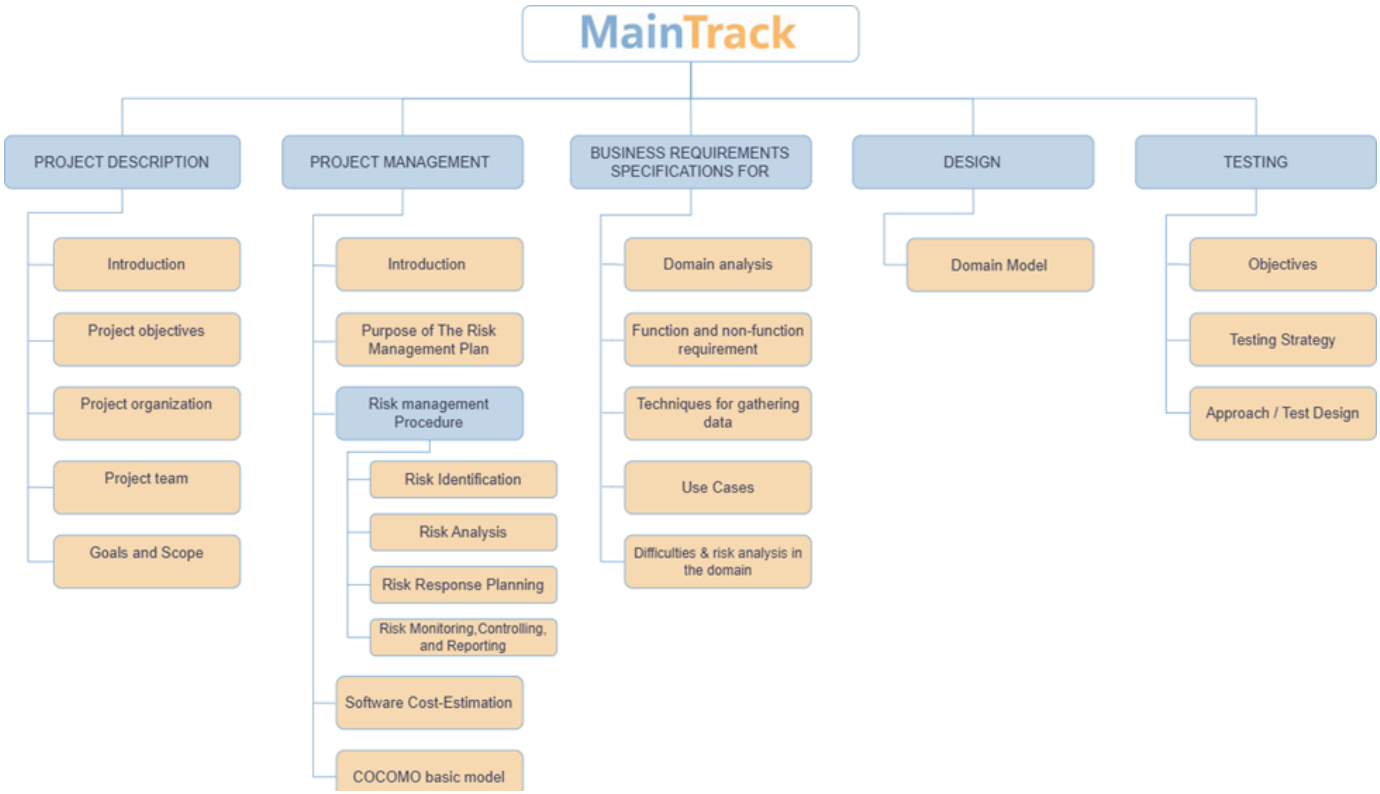


Figure 1/Project organization

2.4 Project team

Student Name	Student ID	Participation
Sarah Algarni	2205539	All team members contributed equally to the development of this project
Lujain Alqarni	2205563	
Shatha Alshaikh	2206240	

2.5 Goals and Scope

2.5.1 Project Goals

- To develop a reliable, scalable, and user-friendly maintenance tracking platform suitable for large institutional environments.
- To significantly reduce maintenance request processing and response times through automation and structured workflows.
- To enhance coordination, transparency, and accountability among maintenance teams and administrative staff.
- To provide accurate, centralized reporting and real-time monitoring of maintenance activities to support informed decision-making.
- To improve overall service quality and user satisfaction within campus facility management.

2.5.2 Project Scope

The scope of the MainTrack system includes the digital management of maintenance operations within university or large facility environments. This encompasses the submission of maintenance requests by authorized users, automated task prioritization and technician assignment, real-time status tracking, and administrative oversight through dashboards and

reports. The system focuses on streamlining communication and improving operational efficiency within the maintenance workflow.

2.5.2.1 Included

- Secure user registration, authentication, and role-based access control.
- Maintenance request submission with detailed issue descriptions, categorization, and precise location specification.
- Automated technician assignment based on request priority, workload distribution, and availability.
- Real-time request status tracking with system-generated notifications for key status changes.
- Real-time fault detection using IoT devices or sensor-based monitoring.
- Administrative dashboard for monitoring requests, managing users, and overseeing technician performance.
- Generation of summary reports and analytics related to maintenance requests, response times, and resolution rates.

2.5.2.2 Excluded

- Execution of physical repair work or on-site maintenance activities.
- Integration with external financial, procurement, or inventory management systems.
- Management of facilities located outside the designated campus or organizational boundaries.
- Advanced predictive maintenance or AI-driven fault analysis features.

3 PHASE 2: PROJECT MANAGEMENT

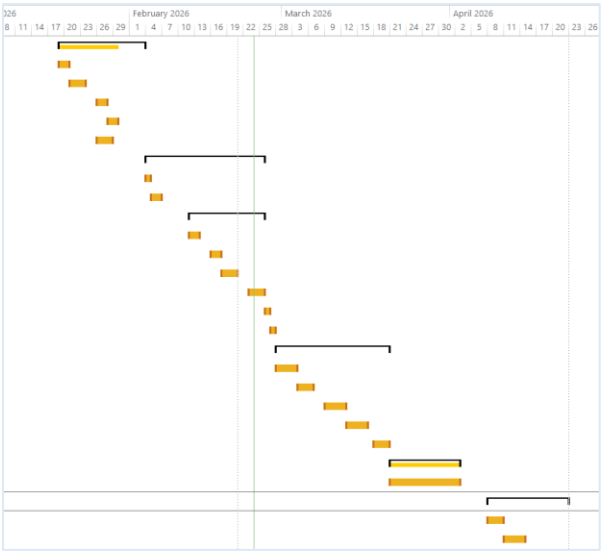
3.1 Introduction

Project management is a repeatable process that takes people to plan, organize, manage resources (especially money and time) over several activities or tasks and project specific deliverables with the end goal of completing an entire project. It is a disciplined way of managing projects that helps teams work efficiently by realizing time, cost and quality. It makes sure that each step of the project was done in accordance with well-established standards and practices.

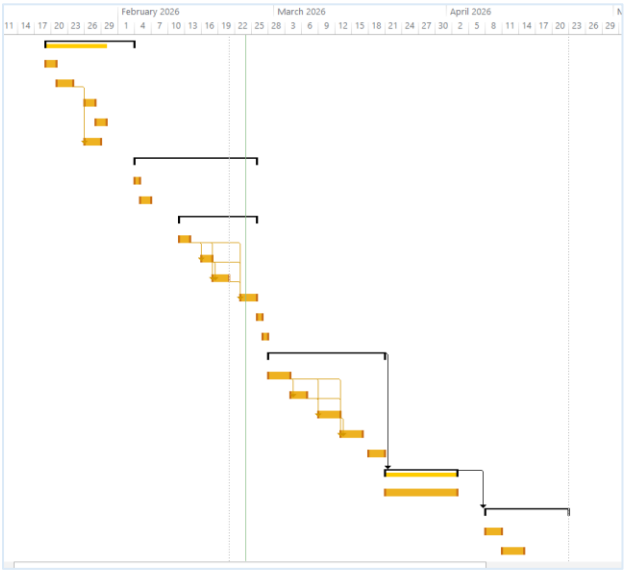
3.1.1 Activities and responsibilities

	Task Mode	Task Name	Duration	Start	Finish	Predecessors
1		**PROJECT DESCRIPTION**	**12 days**	**Mon 1/19/26**	**Tue 2/3/26**	
2		Introduction	2 days	Mon 1/19/26	Tue 1/20/26	
3		Project Objectives	3 days	Wed 1/21/26	Fri 1/23/26	
4		Project Organization	2 days	Mon 1/26/26	Tue 1/27/26	
5		Project Team	2 days	Wed 1/28/26	Thu 1/29/26	
6		Goals and Scope	3 days	Mon 1/26/26	Wed 1/28/26	3
7		**PROJECT MANAGEMEN**	**16 days**	**Wed 2/4/26**	**Wed 2/25/26**	
8		Introduction	1 day	Wed 2/4/26	Wed 2/4/26	
9		Purpose of the Risk I	2 days	Thu 2/5/26	Fri 2/6/26	
10		**Risk Management P**	**10 days**	**Thu 2/12/26**	**Wed 2/25/26**	
11		Risk Identification	2 days	Thu 2/12/26	Fri 2/13/26	
12		Risk Analysis	2 days	Mon 2/16/26	Tue 2/17/26	11
13		Risk Response Plai	3 days	Wed 2/18/26	Fri 2/20/26	11,12
14		Risk Monitoring, C	3 days	Mon 2/23/26	Wed 2/25/26	11,12,13
15		Software Cost-Estimat	1 day	Thu 2/26/26	Thu 2/26/26	
16		COCOMO Basic Model	1 day	Fri 2/27/26	Fri 2/27/26	
17		**BUSINESS REQUIREMEI**	**15 days**	**Sat 2/28/26**	**Fri 3/20/26**	
18		Domain Analysis	3 days	Sat 2/28/26	Tue 3/3/26	
19		Functional and Non-	3 days	Wed 3/4/26	Fri 3/6/26	18
20		Techniques for Gath	4 days	Mon 3/9/26	Thu 3/12/26	18,19
21		Use Cases	2 days	Fri 3/13/26	Mon 3/16/26	18,19,20
22		Difficulties & Risk An	3 days	Wed 3/18/26	Fri 3/20/26	
23		**DESIGN**	**10 days**	**Sat 3/21/26**	**Thu 4/2/26**	**17**
24		Domain Model	10 days	Sat 3/21/26	Thu 4/2/26	
25		**TESTING**	**11 days**	**Wed 4/8/26**	**Wed 4/22/26**	**23**
26		Objectives	3 days	Wed 4/8/26	Fri 4/10/26	
27		Testing Strategy	3 days	Sat 4/11/26	Tue 4/14/26	

3.1.2 Activity bar chart (Organize tasks in a work breakdown structure)



3.1.3 Gantt Chart (Draw dependency constraints)



3.2 Purpose of The Risk Management Plan

The Project Risk Management Plan is intended to anticipate risks so that the project will not be disturbed; rather, the risk should be managed in such away as to facilitate achieving project's. It focuses on ensuring a safe, stable and well-guarded workspace while safeguarding the validity of project information. Furthermore, the plan seeks to protect against financial exposure by instituting checks that keep the project moving along safely and quickly (What Is A Risk Management Plan?, 2024).

3.3 Risk Management Procedure

2.3.1 Process

Project risks are uncertain events that may negatively affect the MainTrack system's schedule, quality, cost, or performance (Bridges, 2023). Because MainTrack depends on digital workflows, automated assignments, dashboards, and IoT-based monitoring, technical and human-related risks must be carefully managed. The risk management process includes four main activities:

- Risk Identification
- Risk Analysis
- Risk Response Planning
- Risk Monitoring and Control

2.3.2 Risk Identification

The following table identifies potential risks specifically related to the **MainTrack smart maintenance system**:

Table 1/ Risk Identification

Risk Type	**Possible Risks**
Technology	The selected cloud hosting service may not support real-time tracking performance when many maintenance requests are submitted simultaneously. (1) IoT sensor devices may fail to transmit accurate fault data to the system dashboard. (2)
People	Lack of experience among developers in integrating IoT modules with web systems. (3) Conflict or poor communication between frontend and backend developers affecting integration. (4)
Organizational	University administration may change priorities, delaying project approval or support. (5) Limited allocated budget may restrict system testing or deployment environment quality. (6)
Tools	Selected development framework may not fully support scalability or security requirements. (7)
Requirements	Maintenance staff may request additional features after system design approval (scope creep). (8) Some functional requirements may be unclear, especially regarding automated technician assignment logic. (9)
Estimation	Underestimation of the time required for testing real-time notifications and dashboard analytics. (10)

2.3.3 Risk Analysis

The following table evaluates each identified risk based on **probability** and **impact on the MainTrack project:**

Table 2/Risk Analysis

Risk	Probability	Effect
(5) Administrative priority change	Low	Catastrophic
(6) Budget limitation	Low	Catastrophic
(1) Cloud performance limitation	Moderate	Serious
(2) IoT sensor malfunction	Moderate	Serious
(7) Framework scalability limitations	Moderate	Serious
(8) Scope creep after approval	High	Serious
(9) Ambiguous assignment logic	Moderate	Serious
(10) Testing time underestimation	High	Serious
(3) Lack of IoT integration experience	Moderate	Tolerable
(4) Developer communication issues	Moderate	Tolerable

2.3.4 Risk Response Planning

Table 3/Risk Response Planning

Risk	Strategy
(5) Administrative priority change	**Accept strategy:** Maintain proper documentation and be prepared to adjust the project timeline if university management decisions change.
(6) Budget limitation	**Avoid strategy:** Plan resource usage carefully and rely on open-source tools and free development platforms to reduce costs.
(1) Cloud performance limitation	**Avoid strategy:** Select a scalable cloud hosting provider that supports load balancing and conduct early performance testing before deployment.
(2) IoT sensor malfunction	**Minimization strategy:** Perform pilot testing for IoT devices before full integration and implement manual reporting as a backup option.
(7) Framework scalability limitations	**Avoid strategy:** Evaluate system requirements carefully before selecting the development framework and test scalability early in development.
(8) Scope creep after approval	**Contingency plan:** Evaluate the impact of any requested changes and require formal approval before modifying the system design.
(9) Ambiguous assignment logic	**Minimization strategy:** Conduct detailed requirement clarification sessions with stakeholders to define technician assignment rules clearly.

(10) Testing time underestimation	**Minimization strategy:** Add buffer time to the project schedule, especially for integration testing and real-time notification validation.
(3) Lack of IoT integration experience	**Minimization strategy:** Allocate time for self-learning, online training, and technical workshops related to IoT integration and API communication.
(4) Developer communication issues	**Minimization strategy:** Conduct weekly coordination meetings and clearly divide responsibilities between frontend and backend teams.

2.3.5 Risk Monitoring, Controlling, and Reporting

Risk monitoring and control ensures continuous tracking of identified risks, newly emerging risks, and the effectiveness of implemented response strategies throughout the MainTrack project lifecycle. The monitoring process includes:

- Maintaining a risk register that documents all risks and their current status.
- Reviewing risks during weekly project meetings.
- Tracking performance indicators such as development progress and testing results.
- Evaluating whether implemented mitigation strategies are effective.
- Reporting major risks or changes to project stakeholders.

If new risks arise particularly related to system performance, IoT integration, or requirement changes they will be analyzed immediately and added to the risk register. Additionally, risk discussions will be included in every project progress meeting to ensure proactive management rather than reactive problem-solving.

3.4 Software Cost-Estimation

3.4.1 Detail Function Point Analysis

1. **Requester**
    - Requester Can Register
    - Requester Can Login
    - Requester Can Logout
    - Requester Can Reset Password
    - Requester Can Submit Maintenance Request
    - Requester Can Edit Submitted Request (Before Processing)
    - Requester Can Cancel Request
    - Requester Can View Request Status
    - Requester Can Receive Notifications
    - Requester Can View Request History

2. **Technician**
    - Technician Can Login
    - Technician Can Logout
    - Technician Can Reset Password
    - Technician Can View Assigned Requests
    - Technician Can Update Request Status

3. **Administrator**
    - Administrator Can Login
    - Administrator Can Logout
    - Administrator Can Reset Password
    - Administrator Can View Dashboard
    - Administrator Can Generate Reports
    - Administrator Can Monitor IoT Alerts

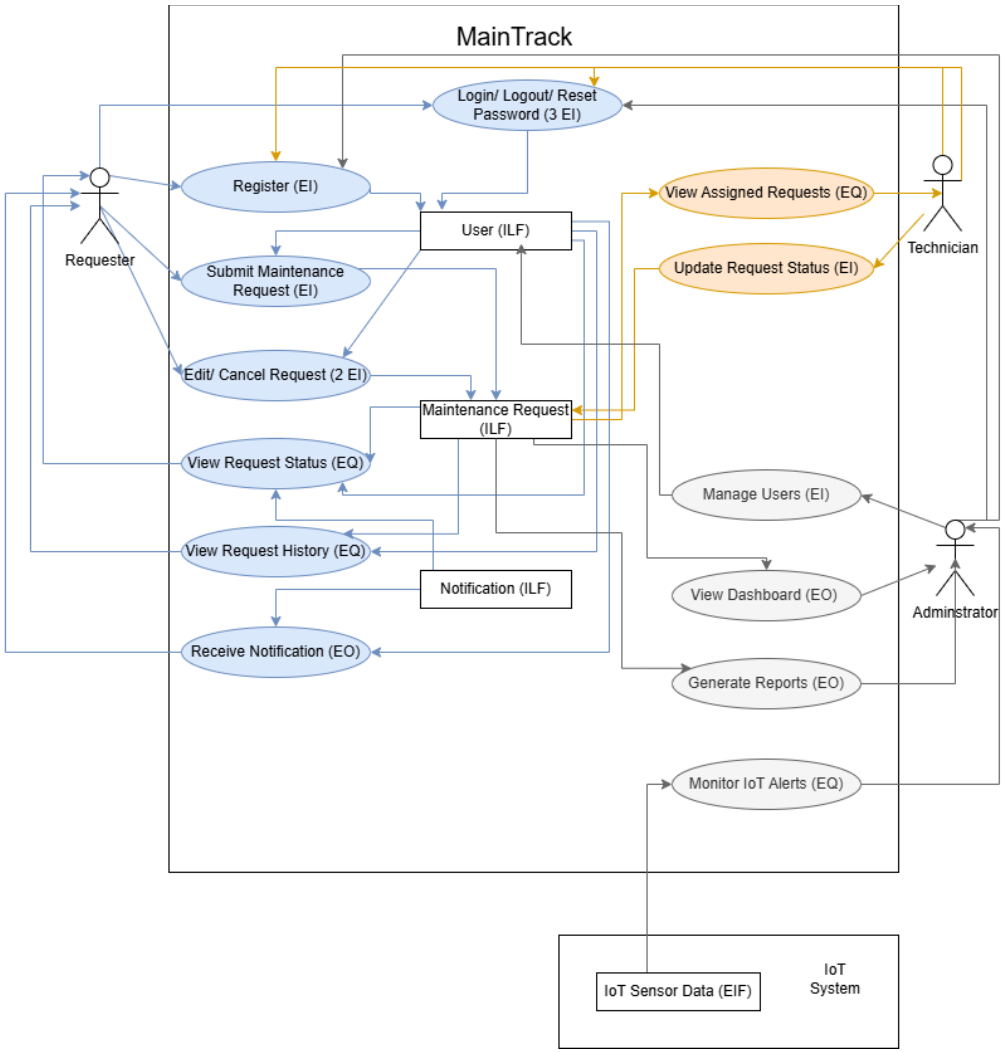


Figure 2/Function Point Analysis

3.4.2 Identity Complexity

Table 4 / Transaction Function Complexity

Transaction Function	Fields / File Involvement	FTRs	DETs
Register Account (EI)	Fields: Name, Email, Phone Number, Password, Confirm Password, Role (Default), Submit File: User	1	7
Login (EI)	Fields: Email, Password, Submit File: User	1	3
Logout (EI)	Fields: Logout Button, Confirm File: User	1	2
Reset Password (EI)	Fields: Email, Verification Code, New Password, Confirm Password File: User	1	4
Submit Maintenance Request (EI)	Fields : Title, Category, Description, Building, Room Number, Attachment, Date , Submit Files: Maintenance Request, User	2	8
Edit / Cancel Request (2 * EI)	Fields: Request ID, Edited fields (title/category/description / Building / Room Number /attachments/ Date), Confirm Files: Maintenance Request, User	2	9
View Request Status (EQ)	Fields: Request ID, Status, Assigned Technician, Last Update, Return Files: Maintenance Request, User	2	5
View Request History (EQ)	Fields: Date Filter, Status Filter, Category Filter, Request List, Return Files: Maintenance Request, User	2	5
Receive Notification (EO)	Fields: Notification Message, Type, Timestamp, Related Request Files: Notification, User	2	4
View Assigned Requests (EQ)	Fields: Technician ID, Assigned Requests list, location, status Files: Maintenance Request, User	2	4
Update Request Status (EI)	Fields: Request ID, New Status, Comment, Submit Files: Maintenance Request, Notification	2	4
Manage Users (EI)	Fields: Create/Edit/Deactivate, role assign, Submit File: User	1	5

View Dashboard (EO)	Fields: Total Requests, Open Requests, Closed Requests, Average Response Time, Return Files – Maintenance Request	1	5
Generate Reports (EO)	Fields: Date Range, Report Type, Total Requests, Resolution Time, Export Option, Submit Files: Maintenance Request	1	6
Monitor IoT Alerts (EQ)	Fields: Device ID, Alert Type, Severity, Location, Timestamp, Return Files: IoT Sensor Data	1	6

Table 5/ Data Function Complexity

Data Function	Fields / File Involvement	RETs	DETs
User (ILF)	Fields: User ID, Name, Email, Phone Number, Password, Role, Status, Created Date	1	8
Maintenance Request (ILF)	Fields: Request ID, Requester ID, Title, Category, Description, Building, Room Number, Attachment, Assigned Technician ID, Status, Preferred Date, Updated Date	2 (1 = Request Information, 1 = Status History)	12
Notification (ILF)	Fields: Notification ID, Receiver ID, Request ID, Message, Type, Timestamp, Status	1	7
IoT Sensor Data (EIF)	Fields: Device ID, Location, Sensor Type, Alert ID, Severity, Reading Value, Timestamp	2 (1 = Device Information, 1 = Alert Records)	7

Table 6/ Transaction Function Complexity

Transaction Function	FTRs	DETs	Complexity	UFP
Register Account (EI)	1	7	Low	3
Login (EI)	1	3	Low	3
Logout (EI)	1	2	Low	3
Reset Password (EI)	1	4	Low	3
Submit Maintenance Request (EI)	2	8	Average	4
Edit / Cancel Request (2 * EI)	2	9	2 x Average	2 x 4 = 8
View Request Status (EQ)	2	5	Low	3
View Request History (EQ)	2	5	Low	3
Receive Notification (EO)	2	4	Low	4
View Assigned Requests (EQ)	2	4	Low	3
Update Request Status (EI)	2	4	Low	3
Manage Users (EI)	1	5	Low	3
View Dashboard (EO)	1	5	Low	4
Generate Reports (EO)	1	6	Low	4
Monitor IoT Alerts (EQ)	1	6	Low	3
Total				54

Table 7/Data Function Complexity

Data Function	RETs	DETs	Complexity	UFP
User (ILF)	1	8	Low	7
Maintenance Request (ILF)	2 (1 = Request Information, 1 = Status History)	12	Low	7
Notification (ILF)	1	7	Low	7
IoT Sensor Data (EIF)	2 (1 = Device Information, 1 = Alert Records)	7	Low	7
Total				28

3.4.3 Unadjusted Function Point Contribution

$$Unadjusted\ Function\ Point\ (UFP) = UFP\ (\text{Transaction Function}) + UFP\ (\text{Data Function})$$
$$= 54\ + 28\ = 82$$

3.4.4 Performance and Environmental Impact

Table 8/General System Characteristics

General System Characteristics	Degree of Influence	Justification
1. Data Communications	4	The system is web-based and depends on continuous communication between client browsers and the server for request submission, status updates, dashboards, and IoT alerts.
2. Distributed Data Processing	2	Processing is mainly centralized on the application server; although users access the system remotely, processing is not heavily distributed across multiple nodes.
3. Performance	4	Real-time request handling, dashboard analytics, and IoT monitoring require responsive system performance to maintain operational efficiency.
4. Heavily Used Configuration	3	The system will be regularly used by requesters, technicians, and administrators, but it does not require specialized hardware or complex infrastructure.
5. Transaction Rate	4	Multiple users may simultaneously submit and update maintenance requests, leading to a moderate-to-high transaction volume.
6. Online Data Entry	4	All user interactions, including registration, request submission, and updates, are performed online through web forms.
7. End-User Efficiency	4	The system is designed to streamline maintenance workflows and improve usability through dashboards and automated notifications.
8. Online Update	4	Maintenance statuses and notifications are updated dynamically and must be reflected immediately in the system.

9. Complex Processing	3	The system includes reporting and data aggregation (e.g., averages and totals) but does not involve highly complex computational algorithms.
10. Reusability	3	Core modules such as authentication, notifications, and reporting can potentially be reused in similar institutional systems.
11. Installation Ease	4	As a web-based system, it requires no client-side installation and can be deployed centrally on a server.
12. Operational Ease	4	The interface is designed to be user-friendly with dashboards and automated workflows requiring minimal technical expertise.
13. Multiple Sites	3	The system can support multiple buildings or departments within the same institution.
14. Facilitate Change	4	Web architecture allows future modifications, scalability, and feature additions with minimal structural impact.
Total	50	

3.4.5 Adjusted Function Point Contribution

$$Value\ Adjusted\ Factor\ (VAF) = \left(0.65\ + (0.01\ \times TDI)\right) = \left(0.65\ + (0.01\ \times 50)\right) = 1.15$$

$$Adjusted\ Function\ Point\ Count = UFP\ \times VAF\ = 82 \times 1.15 = 94.3$$

3.5 COCOMO Basic Model

Table 9/COCOMO Basic Model

Transaction Functions	**Estimation Line of Code**
Register	180
Login	100
Logout	60
Reset Password	120
Submit Maintenance Request	220
Edit / Cancel Maintenance Request	220
View Request Status	150
View Request History	180
Receive Notification	150
View Assigned Requests	200
Update Request Status	200
Manage Users	250
View Dashboard	250
Generate Reports	300
Monitor IoT Alerts	300
Total	2880 **LOC** = 2.88 **KLOC**
Effort (E for organic)	$E = a \times (KLOC)^b = 2.4 \times (2.88)^{1.05}$ $= 7\ Staff - \text{months}$
Time for development (TDEF for organic)	$TDEV = c \times (E)^d = 2.5 \times (7)^{0.38} = 5$ months
Average staff	$Average\ Staff = \frac{E}{TDEV} = \frac{7}{5}$ $= 1.4\ Approx\ 2\ staff$
Productivity	$\text{Productivity} = \frac{LOC}{E} = \frac{2880}{7}$ $= 411\ LOC/staff\_month$

4 PHASE 3: BUSINESS REQUIREMENTS SPECIFICATIONS FOR DISTRIBUTED SYSTEMS

4.1 Domain Analysis

Domain analysis focuses on understanding the operational environment in which the MainTrack maintenance management system will operate. The system is designed for institutional environments such as universities or large facilities where maintenance activities occur frequently and require efficient coordination.

In traditional maintenance processes, requests are often reported manually through phone calls or informal communication. This approach may lead to delays, lack of transparency, and difficulty in tracking the status of maintenance tasks.

The MainTrack system addresses these challenges by providing a centralized digital platform where authorized users can submit maintenance requests, and technicians can receive and manage assigned tasks. The system also integrates IoT-based monitoring to detect faults automatically when possible.

Within this domain, several actors interact with the system including administrators, technicians, and authorized users who submit maintenance requests. The system supports the maintenance workflow by organizing requests, assigning tasks, monitoring progress, and generating reports for administrative decision-making.

4.2 Functional and Nonfunctional Requirements

1. Functional Requirements

1.1 User Registration and Account Management

FR-1.1.1 The system shall allow new users to create an account.

FR-1.1.1.1 The user shall provide a full name during registration.

FR-1.1.1.2 The user shall provide a valid email address.

FR-1.1.1.3 The user shall create a password during registration.

FR-1.1.2 The system shall verify the user's email address during the registration process.

FR-1.1.3 The system shall allow registered users to log in using their email and password.

FR-1.1.4 The system shall allow users to update their profile information.

FR-1.1.4.1 Users shall be able to edit their name.

FR-1.1.4.2 Users shall be able to update their email address.

FR-1.1.4.3 Users shall be able to change their password.

FR-1.1.5 The system shall implement role-based access control for administrators, technicians, and authorized request submitters.

1.2 Maintenance Request Submission

FR-1.2.1 The system shall allow authorized users to submit maintenance requests.

FR-1.2.1.1 The user shall enter a description of the maintenance issue.

FR-1.2.1.2 The user shall select a category for the issue.

FR-1.2.1.3 The user shall specify the location of the issue.

FR-1.2.2 The system shall store submitted requests in the maintenance database.

1.3 Technician Assignment

FR-1.3.1 The system shall automatically assign maintenance requests to technicians.

FR-1.3.1.1 The system shall consider request priority during assignment.

FR-1.3.1.2 The system shall consider technician workload distribution.

FR-1.3.1.3 The system shall consider technician availability.

1.4 Request Status Tracking

FR-1.4.1 The system shall allow users to view the status of their maintenance requests.

FR-1.4.2 The system shall update the request status in real time.

FR-1.4.3 The system shall notify users when the request status changes.

1.5 Fault Detection

FR-1.5.1 The system shall detect faults using IoT sensors or monitoring devices.

FR-1.5.2 Sensor data shall be transmitted to the system dashboard.

FR-1.5.3 The system shall allow technicians to manually report faults if sensors fail to detect or transmit data.

1.6 Administrative Dashboard

FR-1.6.1 The system shall provide an administrative dashboard.

FR-1.6.2 The dashboard shall allow administrators to monitor maintenance requests.

FR-1.6.3 The dashboard shall allow administrators to manage system users.

FR-1.6.4 The dashboard shall display technician performance statistics.

FR-1.6.4.1 The number of tasks assigned to each technician.

FR-1.6.4.2 The number of completed maintenance requests.

FR-1.6.4.3 Maintenance response time statistics.

1.7 Reports and Analytics

FR-1.7.1 The system shall generate summary reports about maintenance requests.

FR-1.7.2 The system shall provide analytics for response times.

FR-1.7.3 The system shall provide analytics for maintenance resolution rates.

2. Non-Functional Requirements

2.1 Performance

NFR-2.1.1 The system shall respond to user requests within 3 seconds under normal operating conditions.

NFR-2.1.2 The system shall support real-time updates for maintenance request status.

NFR-2.1.3 The system shall deliver notifications without significant delay.

2.2 Scalability

NFR-2.2.1 The system shall support a large number of maintenance requests simultaneously.

NFR-2.2.2 The system shall be deployed on a cloud-based infrastructure.

2.3 Security

NFR-2.3.1 The system shall provide secure authentication for all users.

NFR-2.3.2 The system shall enforce role-based access control.

2.4 Reliability

NFR-2.4.1 The system shall maintain continuous operation even if IoT sensor communication fails.

NFR-2.4.2 The system shall preserve all maintenance request data reliably.

2.5 Usability

NFR-2.5.1 The system interface shall support Arabic and English languages.

NFR-2.5.2 The system interface shall be simple and easy for users to navigate.

2.6 Maintainability

NFR-2.6.1 The system shall support structured workflows for maintenance operations.

NFR-2.6.2 The system shall support centralized reporting for maintenance monitoring.

NFR-2.6.3 The system shall be developed using a modern web development framework that supports scalability and security.

2.7 System Availability

NFR-2.7.1 The system shall maintain an availability of at least 99% uptime during normal operating periods.

NFR-2.7.2 The system shall ensure that users can access maintenance request services without frequent interruptions.

2.8 Data Backup and Recovery

NFR-2.8.1 The system shall perform regular backups of maintenance request data.

NFR-2.8.2 The system shall support data recovery in case of system failure or data loss.

4.3 Techniques for Gathering Data

To ensure that the MainTrack system meets the real needs of users and maintenance staff, appropriate techniques were used to gather and analyze system requirements. These techniques helped the project team understand the current maintenance workflow, identify existing problems, and propose suitable system features. In this project, two main techniques were used for data gathering: Observation and Brainstorming Sessions. Observation was used to study current maintenance processes and existing systems, while brainstorming sessions were conducted among the project team to generate ideas and define the key functionalities of the proposed system.

➢ Observation

Observation was used as a technique to understand how maintenance processes are currently performed in institutional environments such as universities and large facilities. Through observing how maintenance issues are reported, processed, and resolved, the project team was able to gain a clearer understanding of the workflow followed by users and maintenance staff. In many cases, maintenance requests are reported through informal communication methods such as phone calls or verbal notifications, which can lead to delays, lack of proper documentation, and difficulty tracking the progress of requests.

In addition to observing current practices, several existing maintenance management systems were reviewed to understand how similar platforms organize maintenance operations. Systems such as IBM Maximo Application Suite, Fiix CMMS, and ServiceNow were examined to observe common features used in maintenance management solutions. These systems typically provide functionalities such as centralized request submission, technician task assignment, real-time status tracking, and administrative dashboards for monitoring maintenance activities. The insights gained from observation helped the project team identify inefficiencies in traditional processes and design a system that supports more organized and transparent maintenance management.

➢ Brainstorming Sessions

Brainstorming sessions were conducted among the project team members to generate ideas and explore possible features for the proposed MainTrack system. During these sessions, the team discussed the challenges identified from the observation stage and collaboratively proposed solutions that could improve the efficiency of maintenance management. Brainstorming allowed the team to evaluate multiple ideas, compare alternative approaches, and identify the most suitable functionalities for the system.

As a result of these discussions, several key system features were defined, including automated technician assignment based on workload and availability, real-time tracking of maintenance requests, system-generated notifications for users and technicians, and an administrative dashboard for monitoring maintenance activities and generating performance reports. The brainstorming process helped ensure that the system design addresses real operational needs and incorporates practical solutions for improving maintenance coordination and transparency.

4.4 Use Case description tables

4.4.1 Register Account

Table 10/ Register Account Use Case description

Use Case Name	Register Account
Actors	User
Triggering Event	User selects “Register Account”.
Scenario	A new user creates an account in the **MainTrack** system by providing the required information and verifying their email address.
Flow of Activities	1. User opens registration page. 2. System displays registration form. 3. User enters full name, email address, and password. 4. User submits registration form. 5. System validates input data. 6. System sends verification email to user. 7. User verifies email address. 8. System activates account and confirms successful registration.
Special Requirements	The system must validate email format and ensure password security requirements.
Pre-condition	The user does not already have an account and has access to the registration page.
Post-condition	A new account is created and verified. The user can log in to the system.
Exception Conditions	E1: Email already exists E2: Missing or invalid input

4.4.2 Submit Maintenance Request

Table 11/ Submit Maintenance Request Use Case description

Use Case Name	Submit Maintenance Request
Actors	User
Triggering Event	User selects Submit Maintenance Request
Scenario	User submits maintenance request describing an issue in a specific location. System stores request and automatically assigns a technician.
Flow of Activities	1. User logs into system. 2. User opens maintenance request page. 3. System displays request form. 4. User enters issue description. 5. User selects issue category. 6. User specifies issue location. 7. User submits request. 8. System validates entered information. 9. System stores request in maintenance database. 10. System assigns request to technician based on priority, workload, and availability. 11. System confirms successful submission.
Special Requirements	System must store requests securely and apply technician assignment logic based on priority, workload, and availability.
Pre-condition	User is logged into system.
Post-condition	Maintenance request is created and assigned to technician.
Exception Conditions	E1: Missing request information → system prompts user to complete field E2: System failure during submission → request not stored.

4.4.3 Manage Users

Table 12/ Manage Users Use Case description

Use Case Name	Manage Users
Actors	Administrator
Triggering Event	Administrator selects **Manage Users** from dashboard
Scenario	Administrator manages system users by viewing and updating user accounts and roles through the administrative dashboard.
Flow of Activities	1. Administrator logs into system. 2. Administrator opens admin dashboard. 3. Administrator selects user management option. 4. System displays list of registered users. 5. Administrator selects specific user account. 6. Administrator updates user information or role. 7. Administrator confirms changes. 8. System validates updates. 9. System saves updated user information.
Special Requirements	Role-based access control must be enforced.
Pre-condition	Administrator is authenticated in system.
Post-condition	User account information or role is updated successfully.
Exception Conditions	E1: Invalid user data → system displays error message. E2: Unauthorized role modification → system rejects update.

4.4.4 Update Request Status

Table 13/ Update Request Status Use Case description

Use Case Name	Update Request Status
Actors	Technician
Triggering Event	Technician selects assigned maintenance request
Scenario	Technician updates maintenance request status. System updates the status in real time and notifies the user.
Flow of Activities	1. Technician logs into system. 2. Technician opens assigned requests list. 3. System displays assigned maintenance requests. 4. Technician selects maintenance request. 5. Technician updates request status. 6. Technician confirms update. 7. System records updated request status. 8. System updates status in real time. 9. System sends notification to user. 10. User views updated request status.
Special Requirements	System must support real-time request status updates.
Pre-condition	Maintenance request is assigned to technician.
Post-condition	Request status is updated and user receives notification.
Exception Conditions	E1: Request not assigned to technician → system denies update. E2: System failure during update → status not changed.

4.5 Difficulties & Risk Analysis in the Domain

The maintenance management domain involves several operational and technical challenges that may affect the effectiveness of the system.

One challenge is the reliability of IoT sensors used for fault detection. Sensors may fail to transmit accurate data due to hardware malfunctions or connectivity issues, which may require manual reporting by technicians.

Another difficulty is the coordination between different users involved in the maintenance process. Miscommunication between administrators and technicians may lead to delays in resolving maintenance issues.

Workload distribution among technicians may also create challenges, particularly when multiple maintenance requests are submitted simultaneously. Ensuring fair task allocation and prioritizing urgent issues is essential for maintaining operational efficiency.

Additionally, technological risks such as cloud system performance or network availability may affect the real-time monitoring capabilities of the system.

Understanding these difficulties helps the development team design appropriate system features to reduce operational risks and improve maintenance management efficiency.

4.6 Use case diagram for the given problem

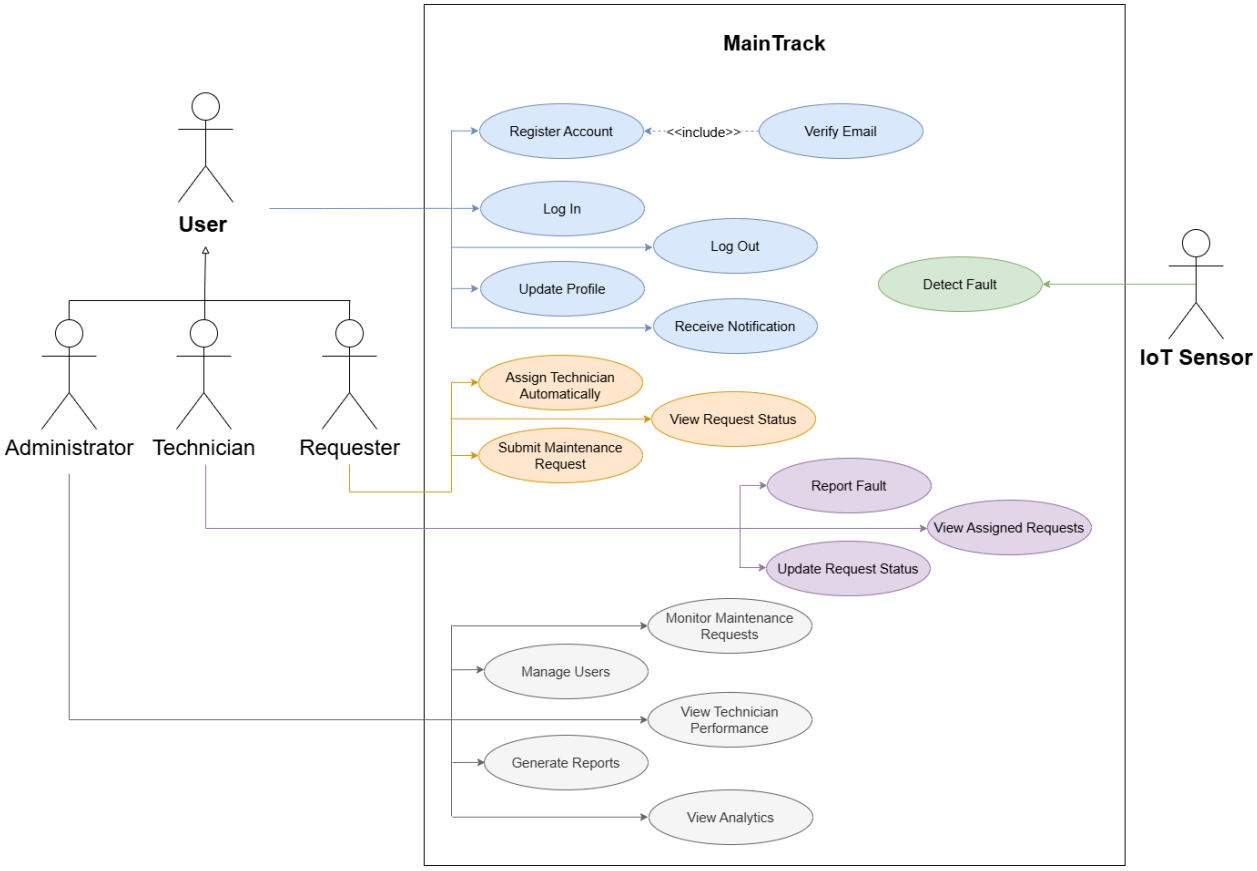


Figure 3/Use case diagram

5 PHASE 4: DESIGN, STRUCTURING, AND MODELING ARCHITECTURES FOR DISTRIBUTED SYSTEMS

5.1 Domain Model

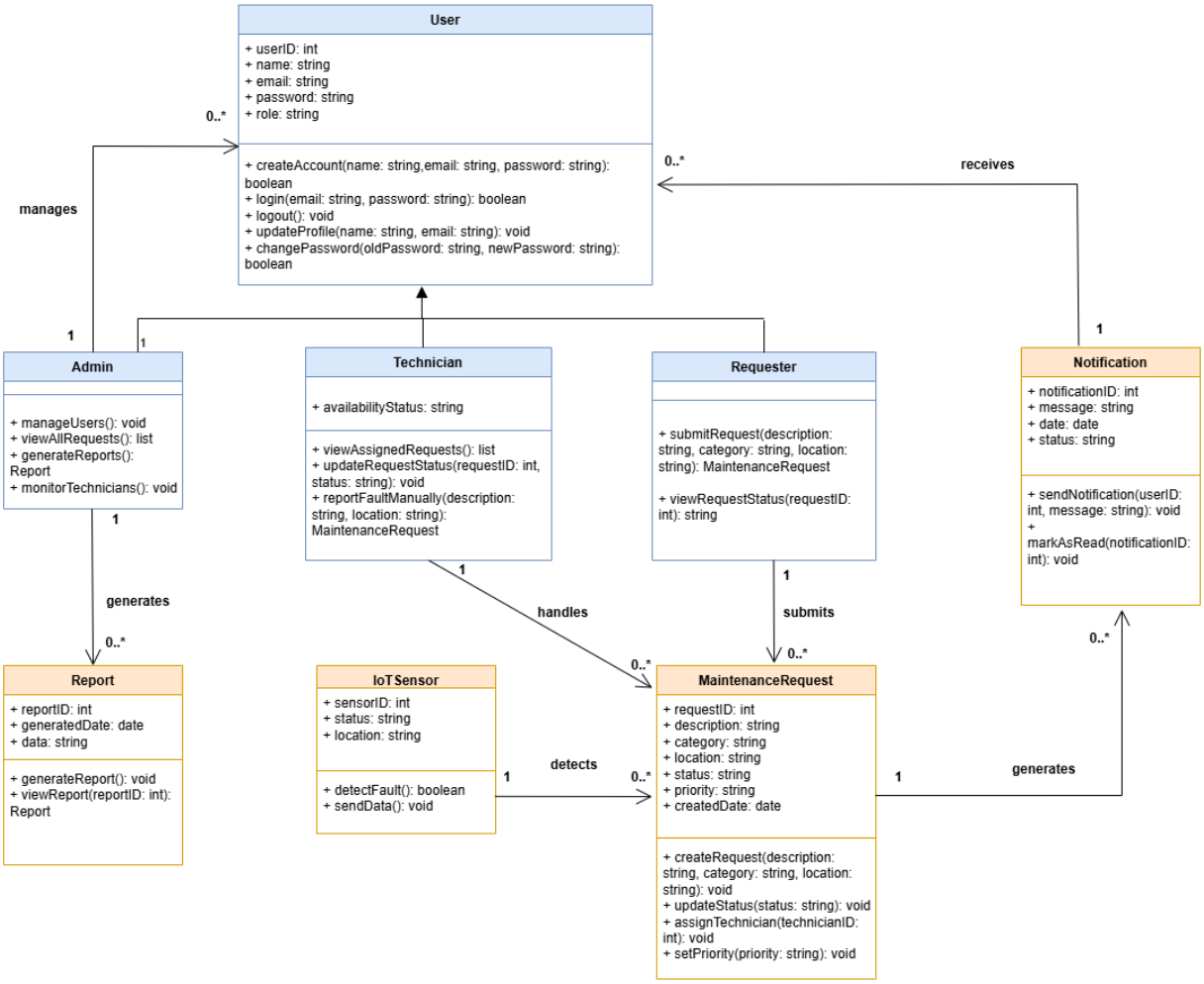


Figure 4 /Domain Model

5.2 Key Challenges in Distributed System Design

The design of the MainTrack distributed system considers several key issues as identified in distributed systems engineering.

1. Transparency
The system is designed to appear as a unified platform for users. Users can submit and track maintenance requests without needing to understand the underlying distributed components such as servers or IoT integrations.

2. Openness
The system is designed using standard web technologies and communication protocols to ensure compatibility and integration with external components such as IoT sensors.

3. Scalability
The system is built to handle an increasing number of users and maintenance requests, especially in large environments such as universities. Cloud-based deployment supports system scalability.

4. Security
The system enforces secure authentication and role-based access control to ensure that only authorized users can access specific functionalities.

5. Quality of Service
The system aims to provide fast response times, real-time request tracking, and reliable notification delivery to ensure a good user experience.

6. Failure Management
The system is designed to handle failures such as IoT sensor communication issues. In such cases, technicians can manually report faults to ensure system continuity.

5.3 Layered Architecture of the System

The MainTrack system follows a layered client-server architecture which organizes the system into distinct layers, each responsible for a specific function (Tanner, 2025).

1. Presentation Layer
This layer represents the user interface of the system. It allows users, technicians, and administrators to interact with the system by submitting maintenance requests, viewing request status, updating tasks, and accessing dashboards.

2. Data Management Layer
This layer is responsible for handling data operations such as retrieving, updating, and organizing system data. It manages interactions between the application logic and the database.

3. Application Processing Layer
This layer contains the core business logic of the system. It processes maintenance requests, handles automatic technician assignments, manages request status updates, and generates notifications.

4. Database Layer
This layer stores all persistent data including user accounts, maintenance requests, system logs, and reports.

This layered structure improves system organization, scalability, and maintainability by separating system responsibilities across different layers.

5.4 SOA / MICROSERVICE ARCHITECTURE

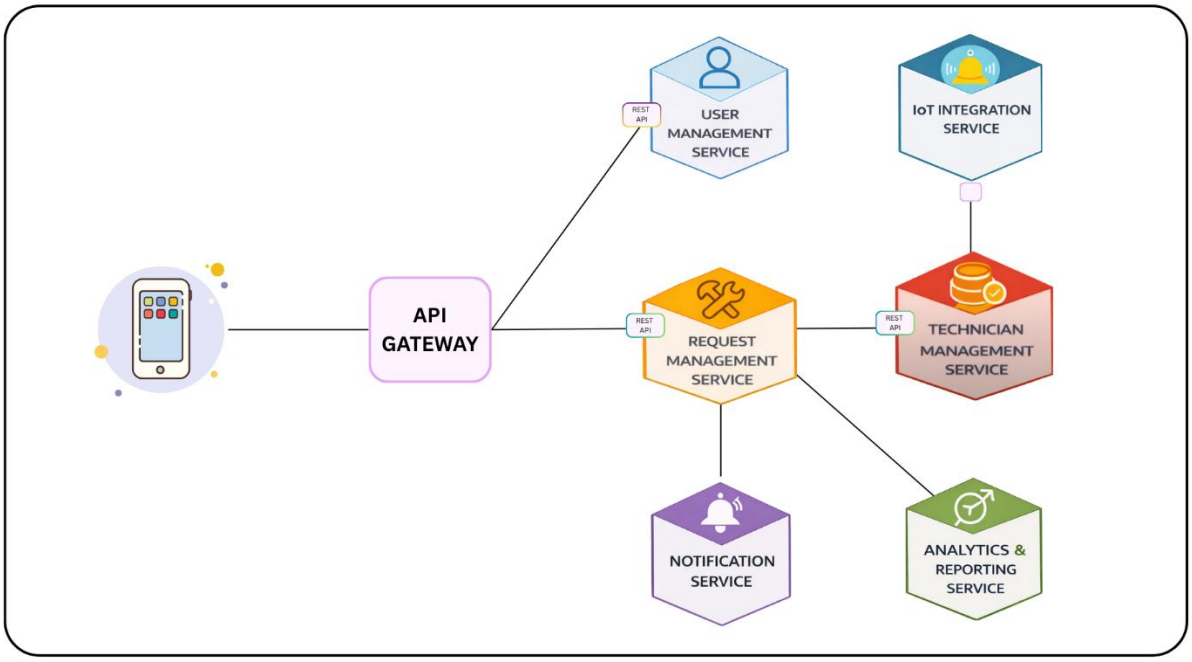


Figure ᵒ /SOA / MICROSERVICE ARCHITECTURE ((generated using AI tool))

- **User Management Service**
  Handles user registration, login, email verification, and profile management.
- **Request Management Service**
  Manages maintenance requests, including submission, updating request status, and storing request details.
- **Technician Management Service**
  Responsible for automatically assigning technicians to maintenance requests based on priority, availability, and workload.
- **Notification Service**
  Sends notifications to users when the status of maintenance requests changes.
- **Analytics and Reporting Service**
  Generates reports and provides analytics related to maintenance performance and technician activities.
- **IoT Integration Service**
  Handles communication with IoT sensors and processes fault detection data.

5.5 Domain-Driven Design (DDD)

Domain-Driven Design (DDD) is a software development approach that focuses on designing systems based on the real business domain and its operational requirements. Instead of organizing the system only around technical layers, DDD organizes the software around business concepts, rules, and workflows. In the MainTrack project, DDD is applied to model the maintenance management operations of universities and large facilities, where users submit maintenance requests, technicians perform repair tasks, administrators monitor activities, and IoT sensors detect faults automatically (Domain-Driven Design (DDD), 2025).

Applying DDD in MainTrack helps reduce complexity, improve communication between stakeholders and developers, and create a system structure that is scalable and easier to maintain.

5.5.1 Core Domain of MainTrack

The core domain represents the most important business area that delivers the main value of the system. For MainTrack, the core domain is Smart Maintenance Request Management, which focuses on managing the complete maintenance process efficiently. This includes receiving maintenance requests, prioritizing issues based on urgency, assigning technicians to tasks, tracking repair progress, detecting faults through IoT devices, sending alerts and notifications, and monitoring overall maintenance performance. This core domain is the primary reason the system exists and the central function around which all other features are built.

5.5.2 Ubiquitous Language

Ubiquitous Language is a common vocabulary shared by developers, administrators, and domain experts. Everyone uses the same terms to avoid confusion.

Table 14/Ubiquitous Language

Shared Term	Meaning in MainTrack
Maintenance Request	A submitted issue requiring repair
Technician	Maintenance staff assigned to tasks
Priority	Urgency level of request
Status	Current progress of request
Fault Alert	Automatically detected issue
Report	Maintenance summary or analytics

5.5.3 Bounded Contexts

A Bounded Context is a logical boundary that separates a large system into smaller focused areas. Each context has its own responsibilities and internal model.

Table 15/Bounded Contexts

Bounded Context	Responsibilities
Request Management Context	Submit, update, and track requests
Technician Management Context	Assign technicians and monitor workload
User Management Context	Manage users and roles
IoT Monitoring Context	Detect equipment faults
Notification Context	Send alerts and updates
Reporting Context	Generate dashboards and reports

5.5.4 Context Map

A Context Map shows how bounded contexts interact with each other within the system. It helps in understanding dependencies and communication flows between different parts of the domain. In MainTrack, the IoT Monitoring Context sends alerts to the Request Management Context whenever faults are detected. The Request Management Context communicates with the Technician Management Context to assign maintenance tasks to available technicians. It also triggers the Notification Context after any request updates so users can be informed. In addition, the Reporting Context collects data from all contexts to generate summaries and performance analytics. For example, when a sensor detects overheating, the IoT Monitoring Context sends the issue to the Request Management Context, which then creates a maintenance request.

5.5.5 DDD Building Blocks in MainTrack

1. Entity

An Entity is an object with a unique identity that remains the same over time even if its attributes change.

Table 16/DDD Entity Building Block

Entity	Role
Admin	Supervises system
Technician	Performs repairs
Requester	Submits requests
MaintenanceRequest	Main repair issue record
Notification	Stores alerts
Report	Stores reports
IoTSensor	Detects faults
A MaintenanceRequest is considered an Entity because it has a unique identity represented by Request ID. For example, Request ID #1 remains the same request throughout its lifecycle, even when its status changes from Pending to Completed.	

2. Value Object

A Value Object is identified by its attributes only and does not need unique identity. It is usually immutable.

Table 17/DDD Building Blocks Value

Value Object	Example
Priority	High, Medium, Low
Status	Pending, In Progress
Location	Building B - Room 204
Category	Electrical
If a maintenance request becomes more urgent, the existing Priority value object (Medium) is not modified directly. Instead, it is replaced with a new Priority value object (High), which preserves immutability and keeps the design consistent.	

3. Aggregate

An Aggregate is a group of related entities and value objects that are treated as one consistency unit to ensure that all related data is managed together. In MainTrack, the main aggregate is the MaintenanceRequest Aggregate, which represents the complete maintenance request and its associated information. This aggregate includes request details, priority level, current status, assigned technician, and related notifications. By grouping these elements together, the system ensures that any change to a maintenance request remains consistent across all related components. For example, all updates related to a request, such as changing the status or assigning a technician, are handled together within the same aggregate.

4. Aggregate Root

The Aggregate Root is the main entity that controls access to all objects within the aggregate and ensures that business rules are enforced consistently. In MainTrack, the MaintenanceRequest acts as the Aggregate Root of the MaintenanceRequest Aggregate. All changes related to the request, such as assigning a technician, updating the status, or changing the priority level, must be performed through the MaintenanceRequest entity. This approach protects data integrity and prevents unauthorized or inconsistent modifications. For example, a technician cannot directly edit internal request data without passing through the MaintenanceRequest object.

5. Service

A Service contains business logic that does not belong naturally to a single entity or value object.

Table 18/Service and Responsibility

Service	Responsibility
Technician Assignment Service	Select suitable technician
Priority Service	Evaluate urgency
Fault Detection Service	Convert sensor alerts to requests
Report Service	Generate analytics
The Technician Assignment Service evaluates available technicians based on specialization, workload, availability, and request location, then assigns the most suitable technician. For example, when an electrical issue is submitted, the service selects an electrician who currently has the lowest workload.	

6. Repository

A Repository manages storing and retrieving aggregates from the database.

Table 19/Repository and Purpose

Repository	Purpose
RequestRepository	Save and fetch requests
UserRepository	Manage users
NotificationRepository	Store alerts
ReportRepository	Retrieve reports
When a new maintenance request is submitted, the system creates a Request aggregate and stores it through the RequestRepository. The repository handles database operations such as inserting the request record and making it available for future retrieval.	

7. Factory

A Factory creates complex objects while hiding creation logic.

Table 20/Factory and Purpose

Factory	Purpose
RequestFactory	Create maintenance requests
NotificationFactory	Create notifications
ReportFactory	Create report objects
When an IoT sensor detects a fault, the system uses RequestFactory to automatically create a maintenance request. The factory applies the required initialization rules, such as setting the default status to Pending, assigning the issue category, and recording the request source as an IoT alert.	

5.5.6 Example of DDD Workflow in MainTrack

A practical example of Domain-Driven Design in MainTrack can be seen when an air-conditioning unit fails. First, the IoTSensor detects an abnormal temperature and sends the data to the system. Then, the Fault Detection Service analyzes the reading and creates issue data representing the detected problem. After that, the RequestFactory generates a new MaintenanceRequest object to record the issue formally. Next, the Priority Service evaluates the situation and marks the request as High priority due to its urgency. The Technician Assignment Service then selects a qualified Air Conditioning Technician to handle the repair. Once all required information is completed, the RequestRepository stores the maintenance request in the database. Finally, the NotificationFactory sends alerts to the responsible technician and relevant administrators. This workflow demonstrates the practical use of DDD components and how they cooperate to solve real maintenance problems efficiently within the project.

6 PHASE 5: TESTING

6.1 Testing Objectives

One of the most important stages in development is testing where you ensure that the MainTrack works properly by performing his roles and functions without any errors before it gets into production. As an integrated system, it is most crucial in confirming that expected behaviors of the intended application exists and provide confidence for all components working together.

This involves testing a case where the system works fine, discovering defects at an early stage so that all requirements are met. Like any system, testing again has a significant role in making sure that services and IoT integration communicate without issues.

To achieve these, this project emphasises testing where:

- Make sure that functional and non-functional requirements, including performance, reliability & stability are fulfilled.
- Provides seamless communication between distributed components like service integration, and IoT..
- Improve overall system quality, reliability, and performance.
- Enhance the user experience for requesters, technicians, and administrators.

For instance, testing a maintenance request done by the user processes properly across multiple components with data being stored in the database and assigned to technician while notifying end-user without any delay or error.

6.2 Testing Strategy for Distributed Systems

Testing the **MainTrack** system requires a well-structured strategy to ensure that all components operate correctly both individually and as part of a distributed environment. Since the system consists of multiple interconnected services, such as request management, technician assignment, notification services, and IoT integration, different levels of testing are applied to validate system functionality, communication, and overall performance.

The testing strategy in this project includes the following levels:

1. Unit Testing

Unit testing is used to validate individual modules separately to ensure that each component performs its specific function correctly without depending on other parts of the system (Susnjara & Smalley, 2025).

For example, the maintenance request module is tested independently to verify that it correctly accepts user input, validates required fields, and stores the request in the database without errors.

Other modules, such as login and notification components, were also tested individually in a similar manner.

2. Integration Testing

Integration testing focuses on verifying the interaction between different system components, which is critical in distributed systems where multiple services must communicate seamlessly (Susnjara & Smalley, 2025).

For example, when a maintenance request is submitted, the system is tested to ensure proper communication between the request management service, technician assignment service, and notification service, so that the request is assigned correctly and the user is notified.

Additional integrations, such as communication with IoT components, were also validated to ensure consistent data flow across the system.

3. System Testing

System testing evaluates the complete system as a whole to ensure that all components work together correctly and meet the specified requirements in real-world scenarios (Susnjara & Smalley, 2025).

For example, a full workflow is tested starting from user login → submitting a maintenance request → automatic technician assignment → updating request status → sending notifications, ensuring that the entire process works smoothly without failure.

Similar end-to-end scenarios were also tested to confirm overall system reliability and performance.

6.3 Test Design Approach

Test cases in the MainTrack system were designed using both black-box and white-box testing approaches to ensure comprehensive validation of the system. These approaches help verify system functionality based on requirements, as well as evaluate the internal logic and structure of the system.

- **Black-Box Testing**

Black-box testing is a technique used to evaluate the system based on requirements and expected outputs without considering the internal implementation. The tester provides inputs and compares the actual output with the expected output to determine whether the system behaves correctly (GeeksforGeeks, 2026).

Table ٢١ / Functional Test Cases

Test Case	Test Input	Expected Output	Actual Output	Result
TC1	Valid username & password	Access granted	Access granted	Pass
TC2	Valid username & invalid password	Error message displayed	Error message displayed	Pass
TC3	Valid maintenance request data	Request stored successfully	Request stored successfully	Pass

TC4	Technician updates request status	Status updated and notification sent	Notification not sent	Fail
TC5	Admin opens dashboard	Data displayed correctly	Data displayed correctly	Pass

Table 22/ Non-Functional Test Cases

Test Case	Scenario	Expected Output	Actual Output	Status
TC1	Multiple users submit requests simultaneously	System handles requests without delay	System handles requests without delay	Pass
TC2	High number of requests (load test)	System remains stable	System remains stable	Pass
TC3	Slow internet connection	System responds correctly	System responds correctly	Pass
TC4	Long system usage	No crash or failure occurs	No crash or failure occurs	Pass
TC5	Response time check	Response within acceptable time	Response delayed	Fail

- **White-Box Testing**

In this project, white-box testing was used to verify the internal logic and execution flow of the MainTrack system.
Instead of only checking inputs and outputs, this approach focused on how the system processes data internally, especially in modules that contain conditions and loops (Susnjara & Smalley, 2025).

The goal was to ensure that:

- All important decision points in the code are tested
- Different execution paths are covered
- The system behaves correctly under various internal conditions

How White-Box Testing was Applied

1. Testing Decision Logic (Login Example)

The login module contains multiple conditions such as checking whether the email exists and whether the password is correct.

Instead of testing only the final result, different internal scenarios were tested:

- Valid email and correct password → login successful
- Valid email and wrong password → error message
- Invalid email → error message

This ensures that all decision paths inside the login logic are executed and verified.

2. Testing Request Validation Logic

The maintenance request module includes validation rules before storing data.

White-box testing was used to verify conditions such as:

- If required fields are empty → request is rejected
- If all data is valid → request is stored successfully

This helped confirm that the validation logic works correctly for both valid and invalid cases.

3. Testing Loop Logic (Technician Assignment)

The system assigns technicians automatically by checking a list of available technicians.

White-box testing was applied to ensure the loop behaves correctly in different situations:

- No technicians available → no assignment occurs
- Only one technician available → assigned directly
- Multiple technicians → system selects the most suitable one

This verifies that the loop executes correctly under different conditions and does not skip or fail.

Table 23/White-Box Test Cases

Test Case	Scenario	Internal Logic Tested	Expected Result	Actual Result	Status
TC1	Valid login	Email & password check	Access granted	Access granted	Pass
TC2	Wrong password	Password condition	Error message	Error message	Pass
TC3	Invalid email	Email existence check	Error message	Error message	Pass
TC4	Empty request fields	Validation condition	Request rejected	Request rejected	Pass
TC5	Valid request submission	Full execution flow	Request stored	Request stored	Pass
TC6	No technician available	Loop skip case	No assignment	Handled correctly	Pass
TC7	One technician available	Single loop iteration	Assigned correctly	Assigned correctly	Pass
TC8	Multiple technicians	Loop selection logic	Best technician selected	Correct selection	Pass

6.4 Component Testing

Component Name :

Maintenance Request Module

Main Function :

This module is responsible for handling the submission of maintenance requests. It allows users to enter request details, validates the input data, and stores the request in the system database.

How it was Tested Individually?

The module was tested using **unit testing** by isolating it from other system components.

The following steps were performed:

- Providing valid request data (title, category, location, description)
- Submitting incomplete or invalid data to test validation rules
- Checking if the request is stored correctly in the database
- Verifying that error messages appear when required fields are missing

Expected Result:

- The system should accept valid inputs and store the request successfully
- The system should reject invalid or incomplete inputs
- Proper validation messages should be displayed

Actual Result:

- Valid requests were stored successfully in the database
- Invalid inputs were correctly rejected
- Validation messages were displayed as expected

6.5 Final Testing Summary

Modules Tested

- User Authentication Module
- Maintenance Request Module
- Technician Assignment Module
- Notification Module
- Dashboard Module

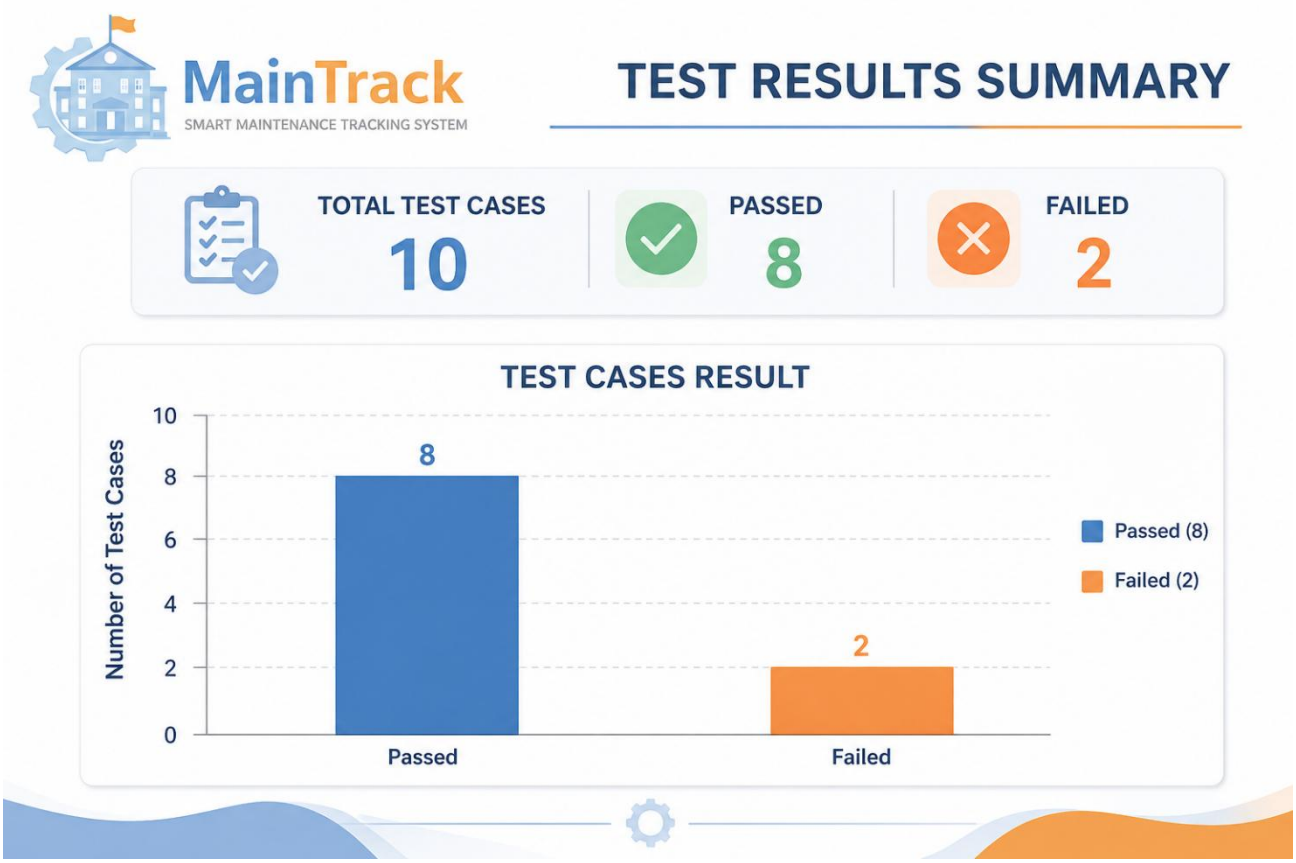


Figure 6/Testing Summary ((generated using AI tool))

Total Number of Test Cases

10 Test Cases

Passed and Failed Test Cases

- Passed: 8
- Failed: 2

Problems Found During Testing

1. Notification was not sent after updating request status
2. System response time was slower than expected under heavy load

How the Problems Were Fixed

- Fixed notification issue by correcting communication between services
- Improved performance by optimizing system processing and response time

7 References

Bridges, J. (2023, Jan 27). *What Is Project Risk? 7 Project Risks to Track*. Retrieved from ProjectManager: https://www.projectmanager.com/training/what-is-project-risk

Domain-Driven Design (DDD). (2025, July 12). Retrieved from GreeksForGreeks: https://www.geeksforgeeks.org/system-design/domain-driven-design-ddd/

GeeksforGeeks. (2026, April 28). *Black Box Testing - Software Engineering*. Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/software-testing/software-engineering-black-box-testing/

Susnjara, S., & Smalley, I. (2025, July 9). *What is software testing?* Retrieved from IBM: https://www.ibm.com/think/topics/software-testing

Tanner, M. (2025, June 18). *Enterprise software architecture patterns: The complete guide*. Retrieved from Function: https://vfunction.com/blog/enterprise-software-architecture-patterns/

What Is A Risk Management Plan? (2024, October). Retrieved from LogicManager: https://www.logicmanager.com/resources/erm/what-is-a-risk-management-plan/